

WebCover: Marginal Coverage Gain-Guided Submodular RL for Efficient Web Testing

Anonymous Authors

Abstract—Automated web GUI testing (AWGT) explores a web application by executing UI events to cover more states and expose potential faults. In practice, testing is usually conducted under a time budget, so redundant interactions waste time that could be used to explore new behavior. Existing RL-based web testing techniques often reduce redundancy through count-decay rewards, but we observe that they can still produce substantial redundant behaviors due to the indirect alignment between such rewards and coverage objective. Besides, these rewards exhibit a submodularity: a newly covered behavior is useful, while repeated coverage has little additional value. This motivates us to use Submodular Reinforcement Learning (SubRL), a framework designed for coverage-oriented sequential decision making. We present **WebCover**, a coverage-oriented SubRL-based web testing approach that aligns the RL objective with the testing objective by using marginal coverage gain as the reward. We train the policy with A2C and use GAE for multi-step advantage estimation. **WebCover** further introduces an uncovered-action-aware mechanism that biases action selection toward unexecuted actions, reducing redundant interactions. We evaluate **WebCover** on ten web applications against representative RL-based baselines. **WebCover** ranks first or second on all subjects for transition coverage and effective-step rate, and on nine subjects for state coverage. Ablation results show that GAE reduces prolonged zero-reward behavior and improves coverage under sparse marginal rewards. The results show that directly rewarding marginal coverage improves transition coverage and effective-step rate on most subjects, reducing redundant exploration under the same testing budget.

Index Terms—web testing, reinforcement learning, coverage, submodular reward

I. INTRODUCTION

Modern web applications implement increasingly complex business logic through dynamic client-side interfaces, server-side state updates, asynchronous rendering, and long user workflows. Testing such applications requires generating sequences of UI actions that can exercise diverse application behaviors and expose hidden failures. This is difficult because a single page may contain many actionable elements, and important behaviors are often reachable only after specific sequences of inputs, clicks, selections, and submissions. As a result, automated web GUI testing (AWGT) is fundamentally a fixed-budget exploration problem: each browser interaction consumes time, and a tester must use a limited interaction budget to cover as much application behavior as possible.

Reinforcement learning (RL) provides a natural framework for this setting, because web testing can be modeled as a sequential decision-making process in which an agent observes a web state, selects an available action, and receives feedback from the resulting page. Recent RL-based web testing systems have shown the promise of this direction. WebExplor uses

curiosity-driven Q-learning and DFA-guided exploration to generate action sequences for web testing [1]. WebQT designs a reward model that encourages the agent to mimic human testers and interact with spatially and functionally related elements [2]. WebRLED further improves representation and action-value learning with deep RL, grid-based action values, an online action discriminator, and episodic/global novelty rewards [3]. These systems demonstrate that adaptive learning can improve web exploration over random or purely model-based strategies.

However, reward design remains a central obstacle for RL-based web testing. In standard RL, the reward function specifies what behavior the agent should learn. If the reward is misaligned with the testing objective, the learned policy may optimize a convenient proxy rather than useful tests. Existing RL-based web testing techniques typically rely on local transition-count curiosity, human-inspired reward indicators, or learned novelty scores. Such signals are useful, but they also make the approach depend on hand-crafted reward components, empirical weights, and application-sensitive engineering choices. More importantly, these rewards often treat the diminishing value of repeated exploration as a heuristic bonus rather than as a property of the testing objective itself.

One might instead try to reward the ultimate testing goal directly: discovering bugs, exceptions, or crashes. In practice, this signal is too sparse and unstable to be the primary learning target for online web testing. Once a failure is found it may not provide enough repeated samples to train a stable policy. Large rewards for failures are useful for ranking or reporting test cases, but they are high-variance signals for learning how to explore. This is why software testing research commonly relies on coverage-oriented objectives, such as state coverage, DOM diversity, transition coverage, and code coverage, as practical surrogates for testing effectiveness.

The key observation of this paper is that coverage-oriented web testing rewards are inherently *submodular*. Visiting a state or executing an interaction for the first time can be valuable, but repeating the same already covered behavior should have little or no additional testing value. In other words, the marginal value of an interaction depends on the exploration history. This violates the usual intuition of a fixed per-step reward $R(s, a)$: the same state-action pair may be useful early in a run and redundant later. Existing curiosity rewards partially capture this phenomenon by reducing rewards as transitions are revisited, but they do not explicitly define the testing objective as a global submodular coverage function.

This view connects web testing to submodular reinforce-

ment learning(subrl), where a trajectory is evaluated by the union of covered elements rather than by an additive sum of state rewards. However, web testing introduces practical challenges absent from standard subrl settings: the action set is state-dependent and dynamically detected, actions are meaningful only after browser execution, and testing budgets often permit only a small number of online interactions. WEBCOVER therefore adopts the submodular marginal-gain objective, while using an online masked actor-critic optimizer suited to variable web action spaces.

The coverage units used by WEBCOVER are deliberately simple: web states and executed transitions. A previously visited page may still contain buttons, links, form submissions, or object-specific actions that have not been executed; conversely, repeatedly executing an already covered action may add no testing value even if the browser interaction is valid. For intuition, one can draw states as nodes and executed actions as arcs, but the formal objective is set coverage over unique states and unique executed transition units.

We propose WEBCOVER, an RL-based web testing framework that optimizes submodular state-action coverage. We define state-action coverage objective in (Section V-B2), which is directly derived from states and transitions set.

To optimize this objective in realistic web applications, WEBCOVER uses a masked online actor-critic policy over the currently available actions. This is necessary because web action spaces are variable: each state exposes a different set of buttons, links, inputs, selectors, and form actions. The actor selects among the valid actions on the current page, while the critic estimates future coverage return. In the raw web state, the marginal reward is history-dependent: the same action may be productive before its coverage unit is covered and redundant afterward. In the augmented coverage state $z_t = (p_t, U_S^t, U_A^t, t)$, however, the reward becomes Markovian.

Hard marginal rewards are principled but sparse. To make them learnable under short online testing budgets, WEBCOVER uses an objective-preserving coverage auxiliary. This auxiliary is derived from the current coverage state and asks whether each visible action still represents an unresolved marginal-coverage opportunity. It is an optimization aid for learning the same submodular coverage objective. In practice, we find that this signal must be paired with a rollout configuration long enough to propagate delayed coverage gains on form-heavy and basin-prone applications.

We evaluate WEBCOVER on multiple open-source and real-world web applications under fixed testing budgets. Our evaluation compares against existing web testing and RL-based baselines, including curiosity-based and reward-engineered approaches. We measure not only the number of discovered states, but also executed transitions coverage, effective-step rate, and when available, code coverage. The results show that explicitly modeling submodular coverage improves exploration efficiency and helps translate limited interaction budgets into more useful web tests.

This paper makes the following contributions:

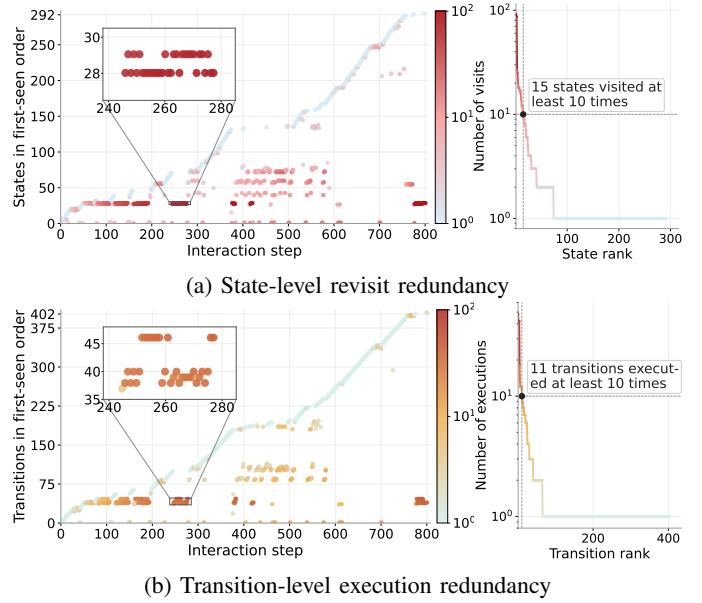


Fig. 1: Exploration progress and redundant interactions of *WebExplor* on GitHub (1 hour). Left: each scatter point denotes the state visited or transition triggered by an interaction, with darker colors indicating higher cumulative counts. Right: rank-frequency plots quantify the repeated visits and executions.

- We formulate RL-based web testing as a budget-constrained online state-transition coverage problem. This formulation makes explicit that common testing metrics are set functions over previously covered elements.
- We propose WEBCOVER, a reinforcement-learning framework that optimizes a submodular coverage objective for web testing. WEBCOVER support online training over variable web action spaces, trains them with strict marginal coverage rewards, and uses an objective-derived coverage auxiliary to improve online credit assignment.
- We conduct an extensive empirical evaluation across benchmark and real-world web applications, showing that WEBCOVER improves coverage, exploration efficiency, and downstream testing outcomes over existing baselines.

II. MOTIVATION

A. Redundant Exploration

Exploration-based AWGT commonly aims to improve exploration coverage within a limited time budget [1], [4], [5], [3], [6]. Under this constraint, redundant interactions consume budget that could otherwise be used to cover new states or transitions. To mitigate such redundancy, recent RL-based web testing techniques often adopt curiosity-driven [7] rewards, which assign higher rewards to less frequently executed actions [4] or transitions [1].

However, when we inspect the traces generated by *WebExplor* [1], a representative curiosity-driven technique, we still observe substantial redundancy. Specifically, we run *WebExplor* on GitHub for one hour and obtain 292 states and 402 transitions within 799 interactions.

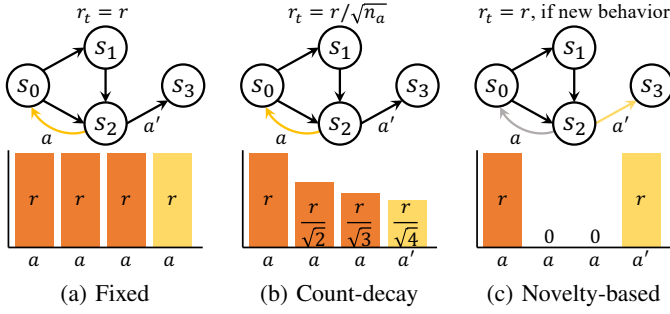


Fig. 2: Comparison of reward signals.

Fig. 1 illustrates the resulting exploration process. In the scatter plots, while new states and transitions continue to appear throughout the run, especially after 600 interactions, many dark points concentrate on early-discovered states and transitions, indicating frequent revisits and repeated executions. For example, both the state-level and transition-level traces show a clear local loop between interactions 240 and 280; similar repeated patterns also appear in the first 200 interactions and again near the end of the run. The rank-frequency plots further show that 15 states are visited at least 10 times, and 11 transitions are executed at least 10 times. In total, the 15 high-frequency states account for 49.6% ($=396/799$) interactions, while the 11 high-frequency transitions account for 32.9% ($=263/799$) interactions.

These observations show that *WebExplor* continues to make exploration progress, but **repeated executions** of already explored behaviors still **occupy a large portion** of the time budget. This motivates the need for mechanisms that explicitly identify and suppress redundant exploration, rather than relying solely on curiosity-driven rewards.

B. Submodularity Behind Existing Rewards

Most existing RL-based GUI testing techniques, including curiosity-driven methods, rely on history-dependent novelty signals to mitigate redundant behavior.

For example, *WebExplor* [1] uses a transition-count decay $1/\sqrt{N(s_t, a_t, s_{t+1})}$ as reward, where (s_t, a_t, s_{t+1}) represents a transition, while QExplore [4] uses an action-frequency decay $1/N(a_t)$ as reward. These designs correspond to the count-decay reward family illustrated in Fig. 2b. Besides, Q-testing [8] maintains a memory of previously visited states and functional scenarios, and assigns high rewards to interactions leading to new functionalities, which more closely resembles the design shown in Fig. 2c. Early work such as AutoBlack-Test [9] assigns rewards based on element-level differences between pre- and post-transition states, yielding fixed rewards for identical transitions (Fig. 2a) and thus easily inducing repetitive back-and-forth state transitions [10].

Although these designs differ in implementation, the effective **reward depends on previously explored behaviors**, resulting in diminishing reward as exploration progresses. Under a coverage-oriented objective, let F measure the coverage of explored behaviors. The reward of executing a behavior e can

be expressed as the marginal gain of F . Specifically, A denote the set of behaviors covered before the first execution of e , and B the corresponding set at a later attempt with additional discoveries, where $A \subseteq B$. The rewards for the first and repeated executions are then given by $F(A \cup \{e\}) - F(A)$ and $F(B \cup \{e\}) - F(B)$, respectively. Intuitively, once e has been covered, its contribution to coverage improvement becomes negligible, resulting in a lower reward:

$$F(A \cup \{e\}) - F(A) \geq F(B \cup \{e\}) - F(B)$$

which is precisely the defining condition of submodularity.

Submodularity behind existing rewards stems from the fact that web testing is fundamentally a coverage-oriented problem aiming to maximize the set of explored behaviors. Motivated by this observation, we reformulate web testing as a Submodular Reinforcement Learning (Section III-A) problem using the marginal gain of a coverage function as the reward, aligning the RL signal with the testing objective.

III. PRELIMINARIES

A. From RL to Submodular RL

A Markov decision process (MDP) [11] is commonly defined as a tuple $\langle S, A, P, R, \gamma \rangle$, where S is the state space, A is the action space, $P(s' | s, a)$ is the transition probability, R is the reward function, and γ is the discount factor. An agent learns a policy $\pi(a | s)$ that maps state to actions to maximize expected cumulative reward.

However, in coverage-oriented tasks, the value of an action depends on previously explored behaviors, motivating a formulation that explicitly models historical exploration. This can be formulated within a **Submodular Reinforcement Learning (SubRL)** framework [12], where the state is augmented with the time step, allowing the model to distinguish the same behavior at different steps.

A submodular MDP is defined as $\langle S, A, P, \rho, H, F \rangle$, where ρ is the initial state distribution and H is the finite horizon, i.e., the step budget. The time-augmented state space is defined as $S = \{0, 1, \dots, H\} \times \mathcal{V}$, where $v \in \mathcal{V}$ is an environment state. Each state $s = (h, v) \in S$ consists of an environment state $v \in \mathcal{V}$ and the current time step h . Let $F : 2^S \rightarrow \mathbb{R}_{\geq 0}$ be a monotone submodular function defined over sets of time-augmented states visited along a trajectory. Given a trajectory $\tau = (s_0, a_0, s_1, a_1, \dots, s_H)$, let $X_h(\tau) = \{s_0, s_1, \dots, s_h\}$ denote the set of time-augmented states visited up to step h . The objective is to learn a policy π that maximizes the expected value of F over the trajectories induced by the policy, i.e., $\max_{\pi} \mathbb{E}_{\tau \sim P^{\pi}} [F(X_H(\tau))]$. Accordingly, the reward obtained at step h is defined as the marginal gain $r_h = F(X_h(\tau)) - F(X_{h-1}(\tau))$.

SubRL also distinguishes Markovian and non-Markovian policies. A Markovian policy conditions on an augmented state that contains the information needed to estimate marginal gains. A non-Markovian policy can condition on the interaction history, for example through recurrent memory.

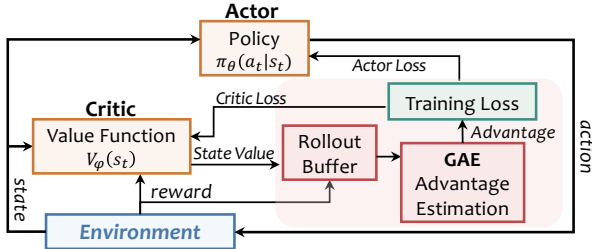


Fig. 3: Standard A2C architecture with GAE.

B. Actor-Critic and GAE

Actor-critic methods are a standard family of policy-gradient algorithms [13]. The actor represents the policy and outputs an action distribution. The critic estimates the value of the current state and serves as a baseline for reducing the variance of policy-gradient updates. Advantage Actor-Critic (A2C) trains the actor and critic synchronously from rollout trajectories collected through environment interaction [14]. Figure 3 illustrates the standard A2C workflow.

In actor-critic learning, the actor is updated according to whether the selected action performs better or worse than the critic’s expectation. The critic is trained to predict future return, and the actor uses this value estimate to adjust the probability of selecting future actions. This makes A2C suitable for learning stochastic policies over sequential decision problems without requiring a fixed tabular value representation.

Generalized Advantage Estimation (GAE) [15] is commonly used with actor-critic methods to compute stable advantage estimates from multi-step rollouts. GAE interpolates between the low-variance but potentially biased TD(0) [16] estimate and the low-bias but high-variance Monte Carlo [17] estimate. It uses information from multi-step rollouts to estimate the advantage of each action, allowing delayed rewards to contribute to the updates of preceding actions. This is useful for coverage-oriented web testing: an action may not immediately increase coverage, but it may lead to later states where new coverage becomes available. GAE provides a standard credit-assignment mechanism for propagating such delayed coverage gains back to the actions that enabled them.

IV. PROBLEM FORMULATION

Given a web application under test, GUI testing interacts with its webpages to cover more states and increase the chance of exposing faults. At each step, the test agent observes the current page, selects and executes an available UI event, and reaches the next state, forming an executed transition; the process then repeats. In practice, testing is usually conducted under a time budget, so coverage needs to be improved efficiently; repeated states or transitions consume budget that could be used to explore new behavior. Therefore, web testing task can be modeled as a coverage problem and formulated as a submodular MDP $\mathcal{M}_{web} = \langle \mathcal{S}, \mathcal{A}, P, \rho, H, F_{cov} \rangle$ [12].

Definition 1 (Time-augmented State): A web state $v \in \mathcal{V}$ represents the current webpage as observed by the agent,

keeping the features relevant to interaction. The concrete representation is described in Section V-B. Following the submodular MDP formulation, the state space is time-augmented as $s_h = (h, v_h) \in \mathcal{S}$, where h is the current interaction step and v_h is the abstract web state observed at that step.

Definition 2 (Action): An action $a \in \mathcal{A}$ represents an available UI event, such as clicking a button and entering text.

Definition 3 (Reward): After executing an action, the agent receives a reward $r_t = F_{cov}(\tau_{t+1}) - F_{cov}(\tau_t)$ based on the coverage gain. Unlike reward signals based on count-decay [1], [4], our reward is directly tied to the coverage objective, as detailed in Section V-B2.

V. APPROACH

A. Overview

To improve coverage of web testing tools under a time budget, we focus on two complementary aspects: (1) learning which behaviors are likely to lead to new coverage; and (2) reducing redundant interactions. To this end, we propose **WEBCOVER**, a coverage-oriented web testing framework.

As shown in Figure 4, we adopt a Submodular Reinforcement Learning (SubRL) formulation, where the reward is defined as the strict marginal gain of a coverage objective, corresponding to exploitation (Section V-B). To reduce redundant interactions, we explicitly encourage the discovery of previously unexecuted behaviors, corresponding to exploration (Section V-C). To jointly optimize these two objectives, we train the SubRL agent using an A2C framework with GAE, which enables learning from multi-step trajectories.

B. Coverage-Oriented AWGT with SubRL

1) State-Action Representation: Given a webpage, **WEBCOVER** first extracts its rendered HTML and identifies the available UI events, which serve as candidate actions for the policy. **WEBCOVER** then builds an input matrix for the policy network, where each row represents one available UI event on the current page. To keep the matrix at a fixed size, pages with fewer events are padded with empty rows, which are excluded during action selection.

To help the policy choose interactions that are more likely to improve coverage, **WEBCOVER** abstracts each available UI event using page-level, event-level, and history information.

- *Page-level context* captures the HTML tag structure, including their general depth in the DOM tree. It also summarizes the available UI events on current webpage, such as the total number of actions and the counts of submit, input, and select events.
- *Event-level information* describes the candidate event itself, including its text, type, DOM position, navigation target. It also records whether the event is inside a form or submits a form.
- *History information* summarizes the local coverage history of the candidate event and the progress of the testing process. It includes the event’s execution count from the current state, zero-gain history, and the execution-count distribution of actions in the reached state if the

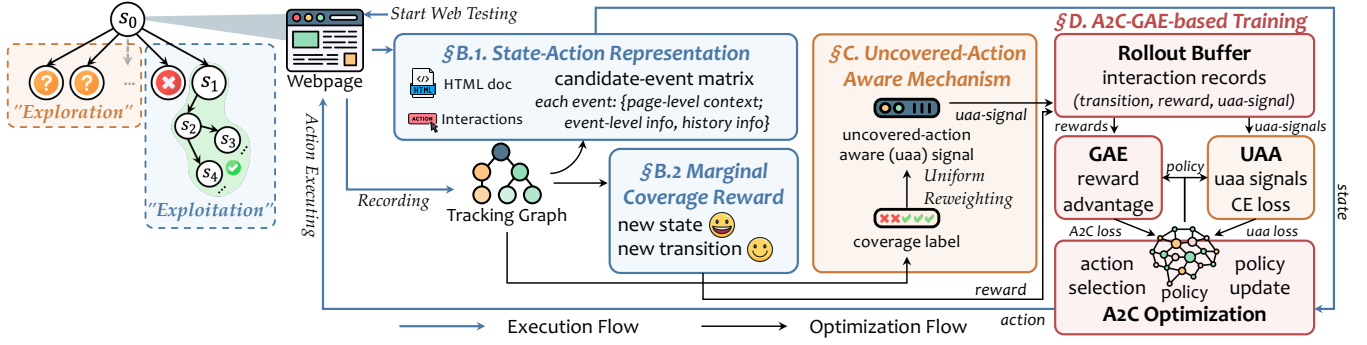


Fig. 4: Overview of WEBCOVER

event has been executed before. It also records recent marginal rewards, remaining budget, coverage progress, and uncovered behavior around the source or possible target, without passing the full covered set to the network.

Together, these features help the policy identify actions that are likely to lead to new states or transitions under the remaining budget.

2) *Marginal Coverage Reward*: WEBCOVER uses the reward to directly encode the testing goal: covering more states and transitions. During testing, WEBCOVER maintains a tracking graph whose nodes are visited web states and whose edges are available UI events. An executed edge points to the state reached after executing the event. An unexecuted edge keeps an unobserved target placeholder and is marked as uncovered.

Let U_S^t be the set of covered abstract states and U_T^t be the set of covered transitions after t interactions. The coverage objective is

$$F_{cov}(t) = |U_S^t| + \alpha|U_T^t|.$$

The learning reward is the marginal gain:

$$r_t = F_{cov}(C_{t+1}) - F_{cov}(C_t).$$

as coverage state $C_h = (U_S^h, U_T^h)$ records what has been covered after h interactions. Thus, a newly covered state or action transition contributes once, while a repeated transition contributes zero. Self-transitions are treated in the same way: the first execution of (s, a, s) can cover a transition, but later executions of the same triple have no additional coverage value. Reset-like actions may receive zero weight when they only control the browser session and do not represent new application behavior.

This reward aligns step-level learning with the final coverage metric. With an undiscounted finite horizon, the sum of marginal rewards corresponds to the final coverage gain. However, the reward is also sparse and history-dependent: a valid browser action is not necessarily coverage-increasing under the current coverage history, and the same action can be useful before its transition is covered but redundant afterward.

C. Uncovered-Action Aware Mechanism

The marginal reward is available only after executing an action. Before execution, WEBCOVER uses the current coverage

record to provide a lightweight exploration signal over visible actions.

For each candidate action $a_t^i \in A_t$, WEBCOVER computes a stable action signature $\psi(a_t^i)$ and checks whether the source-action prefix $(s_t, \psi(a_t^i), *)$ has been resolved in the current run. If no transition with this source state and action signature has been observed, the action is marked as uncovered; otherwise it is marked as resolved.

These labels form an uncovered-action target over the current candidate set. Let $\tilde{y}_t^i = 1$ if a_t^i is uncovered and $\tilde{y}_t^i = 0$ otherwise. If at least one candidate is uncovered, WEBCOVER normalizes the positive entries:

$$y_t^i = \frac{\tilde{y}_t^i}{\sum_j \tilde{y}_t^j}.$$

For example, raw labels $[0, 1, 1]$ become $y_t = [0, 0.5, 0.5]$. If all candidate actions have already been resolved, the auxiliary target is inactive for this step.

The target y_t is stored in the rollout buffer and used by the uncovered-action auxiliary loss during training. It does not change the valid-action mask, the sampled action, or the marginal coverage reward.

D. A2C-GAE-based Training

The marginal coverage reward tells the agent what should be optimized, but an algorithm is still needed to reduce redundant behavior while learning from sparse feedback. WEBCOVER therefore uses a masked actor-critic policy with two complementary training signals. As shown in the 4, the first signal represent Exploitation and the second represent Exploration.

The first signal is the standard actor-critic signal computed from realized marginal coverage rewards. Let $z_t^\pi = \langle X_t, m_t, c_t \rangle$ denote the policy input used for action selection. Given a rollout, GAE propagates future coverage gains back to earlier actions in the same rollout:

$$\hat{A}_t = \sum_{l \geq 0} (\gamma \lambda)^l (r_{t+l} + \gamma V_\phi(z_{t+l+1}^\pi) - V_\phi(z_{t+l}^\pi)).$$

If an action repeatedly serves as a bridge to uncovered states or action transitions, it receives positive advantage and its probability increases in similar coverage-conditioned contexts. This signal supports exploitation: the policy learns to reuse

Algorithm 1 WEBCOVER Online Training

Require: Web application env , root URL u_0 , budget T

```
1: Initialize actor  $\pi_\theta$ , critic  $V_\omega$ , coverage record  $C$ , and
   rollout buffer  $\mathcal{B}$ 
2: Open  $u_0$  and observe the initial browser page  $p$ 
3: repeat
4:    $s, A, z, y \leftarrow \text{PREPROCESS}(p, C) \triangleright z$ : policy input,  $y$ :
     uncovered-action target
5:   Sample an executable action  $a \sim \pi_\theta(\cdot | z)$ 
6:   Execute  $a$  in the browser and observe next page  $p'$ 
7:    $s' \leftarrow \text{ABSTRACTSTATE}(p')$ 
8:   Resolve the executed transition  $e = (s, a, s')$ 
9:    $r \leftarrow F_{cov}(C \cup \{s', (s, a, s')\}) - F_{cov}(C)$ 
10:  Store  $(z, a, r, y)$  into rollout buffer  $\mathcal{B}$ 
11:  Update coverage record  $C \leftarrow C \cup \{s', (s, a, s')\}$ 
12:   $p \leftarrow p'$ 
13:  if  $\mathcal{B}$  reaches rollout length or  $t = T$  then
14:    Compute GAE advantages from rewards in  $\mathcal{B}$ 
15:    Update actor and critic with masked A2C
16:    Apply UAA auxiliary loss using active targets in
        $\mathcal{B}$ 
17:    Clear  $\mathcal{B}$ 
18:  end if
19: until time budget exhausts
```

actions that have empirically led to downstream coverage. By using GAE method, the actor loss uses the sampled action and its realized downstream coverage return to update the actor, which is defined as:

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}_t \left[\log \pi_\theta(a_t | X_t, m_t, c_t) \hat{A}_t \right].$$

Here X_t , m_t , and c_t are the candidate-action matrix, valid-action mask, and coverage summary. The term $\hat{A}_t$ is the GAE advantage computed from the marginal coverage rewards.

The second signal is the uncovered-action auxiliary signal introduced in Section V-C. For each training step, y_t denotes the normalized target distribution over currently visible uncovered actions. The auxiliary loss trains the actor distribution to allocate probability mass to these unresolved candidates:

$$\mathcal{L}_{\text{UAA}} = -\mathbb{E}_t \left[\sum_i y_t^i \log \pi_\theta(a_t^i | X_t, m_t, c_t) \right].$$

This loss is applied to the same actor distribution used for action selection. It does not introduce a separate policy, nor does it alter the marginal reward. Instead, it provides a local exploration signal that complements the actor-critic signal computed from realized coverage gains.

Algorithm 1 summarizes the online training loop of WEBCOVER. Lines 1–2 initialize the actor, critic, coverage record, rollout buffer, and browser page. At each interaction, Line 4 pre-processes the current page into an abstract state, visible candidate actions, policy input z , and uncovered-action target y . Lines 5–7 sample one executable action, execute it in the browser, and abstract the next state.

Coverage is counted after execution. Lines 8–10 resolve the executed transition, compute the marginal gain of F_{cov} , store the interaction in the rollout buffer. Lines 11–12 update the coverage record and advance the browser page.

When the rollout buffer reaches the rollout length or the run ends, Lines 14–17 perform online training. GAE computes advantages from the marginal rewards in the buffer, masked A2C updates the actor and critic, and the UAA loss is applied to rollout items whose uncovered-action targets are active. The UAA loss only affects training; it does not change the action mask or the marginal coverage reward.

VI. EXPERIMENTAL SETUP

A. Baselines

To evaluate WEBCOVER, we compare it with several representative and competitive baselines from automated web testing and RL-based web exploration.

- **WebExplor** [1] is a curiosity-driven RL approach for automated web testing. It uses transition-frequency curiosity and DFA during online testing to guide exploration.
- **QExplore** [4] is a Q-learning-based exploration strategy for dynamic web applications. It uses guided search and execution-count-based signals to improve exploration.
- **WebQT** [2] is a Q-learning-based web testing approach. It combines state abstraction, action extraction to designed reward model.
- **WebRLED** [3] is a deep reinforcement learning approach for automated web GUI testing. It extracts states and actions from HTML documents and uses a grid-based representation with DQN to guide action selection.

B. Research Questions

RQ1 (Tool Performance): Under the same time or interaction budget, does WEBCOVER improve state/action coverage (F_{cov}), exploration efficiency, code coverage, and generalizability over existing web testing baselines?

RQ2: (Ablation for GAE). Does the GAE mechanism and the A2C rollout configuration improve learning efficiency without altering the hard marginal reward or action mask?

RQ3: (Reward design) Holding the state/action representation and network architecture fixed, is the hard marginal coverage reward more effective than local count-decay curiosity or manually weighted reward proxies?

C. Benchmarks

We evaluate WEBCOVER on a set of benchmark and real-world web applications that exercise different exploration patterns: compact CRUD applications, form-heavy workflows, collaboration and project management systems, content-oriented applications, and larger enterprise-style applications. Table I summarizes the benchmark suite. Each run starts from the configured root page of the application and is executed under the same fixed interaction or time budget. The browser state is reset between independent runs, but the policy is trained online within each run; no offline navigation model of the target application is provided to the agent.

TABLE I: Benchmark applications used in the evaluation.

Application	Category	Exploration characteristics
4ga Boards	Forum / board	Many object pages and repeated local actions.
Agilefant	Project management	Workflow-heavy CRUD and nested entities.
GitHub	Collaboration	Large link/action surface with list-detail flows.
Gadael	Benchmark web app	Deep navigation regions and repeated local basins.
Nextcloud	File collaboration	Dense UI actions and repeated object operations.
Odoo	Business management	Enterprise-style modules and form workflows.
PetClinic	CRUD benchmark	Small but structured navigation and forms.
RealWorld	Social / content	Compact state space with many content actions.
SplittyPie	Finance / sharing	Form-centered workflows and recurring pages.
TimeOff	HR management	Calendar/request workflows and admin actions.

D. Metrics

We report both optimized coverage metrics and external testing outcomes. Coverage metrics include F_{cov} , *Unique States*, *Unique Transition*, *effective rate*. In the result tables, *Unique States* is defined by url and action sets. *Transitions* means the number of distinct resolved state-action-state triples executed during the run, $|\{(s_t, a_t, s_{t+1})\}|$. We define *effective_rate* as the ratio of *Transitions* to *Interactions*.

External outcomes include code coverage when instrumentation is available. We treat these as downstream testing outcomes: the primary learning objective is state/action coverage, while code coverage measure whether improved behavior exploration translates into conventional testing benefits.

E. Configuration and Implementation

All methods are evaluated under a fixed-budget online testing setting: each run uses a one-hour wall-clock budget and a hard cap of 2,200 browser transitions. For WEBCOVER, candidate actions are padded to at most $A_{max} = 512$ slots, with invalid slots masked out during action selection. The coverage objective uses unit weight for states and resolved transitions with $\alpha = 1$. The main optimizer is masked A2C with undiscounted marginal coverage reward ($\gamma = 1.0$), GAE with $\lambda = 0.98$, and rollout length 64.

WEBCOVER is implemented as a web testing tool that drives Chrome, using Selenium 4.22.0 and PyTorch 2.6.0. All experiments were run on a desktop device equipped with an Intel i7-13700 CPU and 32GB RAM. The device was connected through 1Gbps Ethernet to ensure stable network conditions.

VII. EVALUATION

We design the evaluation to answer whether the submodular coverage objective improves web testing under fixed budgets, and whether the coverage-residual auxiliary improves learning without changing the reward.

A. RQ1: Fixed-Budget Testing Effectiveness

1) *Result*: Tables II reports the unique-state and unique-transition coverage achieved by each approach under the same testing budget. Overall, WEBCOVER demonstrates consistently strong coverage performance across the ten web

applications. For unique-state coverage, WEBCOVER ranks first on three applications and second on six, placing among the top two approaches on nine out of ten applications. For unique-transition coverage, WEBCOVER ranks first on five applications and second on the remaining five, achieving a top-two ranking on all ten applications. These rankings indicate that WEBCOVER performs consistently well across applications with different structures and interaction patterns.

Compared with WebExplor, a representative count-decay reward baseline, WEBCOVER achieves higher mean coverage on all ten applications for both unique states and unique transitions. We further investigate whether WEBCOVER uses the fixed testing budget more effectively and whether its exploration performance is reflected in downstream code coverage. Table III and Table IV complement the coverage results with effective-step-rate and downstream code-coverage measurements, showing that WEBCOVER uses the testing budget effectively.

2) *Statistical Analysis*: We further assess whether the observed performance differences between WEBCOVER and the baseline approaches are statistically significant. We use each application as the unit of analysis. For each application and approach, we first aggregate the five independent runs using their mean and then compare WEBCOVER with each baseline using a two-sided Wilcoxon signed-rank test. We apply the Holm correction to account for multiple comparisons within each metric and regard a Holm-adjusted p -value below 0.05 as statistically significant. We additionally report the matched-pairs rank-biserial correlation as the effect size, where a positive value favors WEBCOVER.

The statistical results are consist with the coverage trends reported in Table II. For unique state coverage, WEBCOVER significantly outperforms WebExplor ($p_{Holm} = 0.008$, effect size = 1.00). After Holm correction, the differences from WebRLED, QExplore, and WebQT do not reach statistical significance, although WEBCOVER achieves higher mean state coverage on 6/10, 8/10, and 8/10 applications, respectively. These results suggest that the relative advantage of WEBCOVER in unique-state coverage varies across applications and baseline exploration strategies.

The evidence is stronger for unique transition coverage. WEBCOVER significantly outperforms WebRLED ($p_{Holm} = 0.012$, effect size = 0.96), QExplore ($p_{Holm} = 0.020$, effect size = 0.89), and WebExplor ($p_{Holm} = 0.008$, effect size = 1.00). For effective-step rate, WEBCOVER significantly improves over WebExplor and WebQT, both with $p_{Holm} = 0.008$ and effect size = 1.00. Taken together, these results show that the advantage of WEBCOVER extends beyond unique-state coverage: it also discovers more distinct transitions and uses a larger proportion of the testing steps effectively.

Answer to RQ1: Under the same testing budget, WEBCOVER consistently achieves strong state and transition coverage, improves the effective-step rate, and remains competitive in code coverage across diverse web applications.

TABLE II: Coverage results measured by unique states and unique transitions. Values are reported as mean (standard deviation).

Application	Unique States (#)					Unique Transitions (#)				
	WebExplor	QExplore	WebQT	WebRLED	WEBCOVER	WebExplor	QExplore	WebQT	WebRLED	WEBCOVER
4ga Boards	275.4 (24.8)	65.8 (0.4)	402.2 (52.2)	108.0 (52.1)	434.6 (121.3)	349.6 (24.3)	282.2 (5.8)	650.2 (217.5)	379.8 (182.6)	720.6 (54.4)
Agilefant	66.0 (28.6)	20.8 (3.3)	214.8 (30.6)	199.4 (21.4)	188.4 (21.7)	183.8 (49.7)	205.0 (33.0)	740.2 (30.5)	373.6 (25.7)	788.6 (39.7)
GitHub	306.0 (77.8)	113.6 (50.9)	385.6 (16.8)	256.6 (89.2)	432.4 (25.4)	429.4 (94.6)	230.4 (94.1)	760.4 (7.6)	530.0 (346.0)	645.0 (67.1)
Gadael	50.4 (21.6)	18.4 (0.9)	350.2 (24.2)	77.4 (25.3)	214.0 (66.9)	133.0 (25.6)	59.0 (4.2)	814.6 (27.9)	167.4 (62.2)	659.4 (50.9)
Nextcloud	156.4 (45.0)	23.2 (1.1)	218.4 (32.5)	199.0 (20.0)	261.6 (52.5)	265.2 (48.8)	95.8 (1.3)	358.2 (48.5)	262.6 (9.8)	515.0 (29.0)
Odoo	195.2 (41.9)	55.6 (1.7)	165.2 (40.0)	276.6 (163.4)	272.6 (35.8)	284.6 (62.3)	183.4 (33.0)	315.4 (49.2)	400.6 (275.2)	369.8 (38.5)
PetClinic	10.0 (6.4)	7.0 (0.0)	56.0 (7.8)	139.2 (48.2)	97.8 (7.7)	33.6 (4.0)	39.0 (0.0)	189.4 (33.6)	213.2 (68.0)	373.4 (52.2)
RealWorld	22.4 (4.4)	84.4 (8.4)	39.4 (1.9)	30.8 (9.1)	40.2 (1.5)	62.6 (3.2)	479.0 (30.0)	449.8 (22.5)	311.0 (199.5)	463.2 (14.5)
SplittyPie	25.6 (7.2)	2.8 (0.4)	167.0 (21.0)	278.8 (49.6)	183.2 (21.0)	89.2 (11.8)	9.6 (0.9)	493.6 (19.2)	478.6 (72.7)	602.0 (43.5)
TimeOff	23.4 (4.0)	384.6 (6.5)	104.4 (11.3)	118.2 (33.8)	118.4 (12.4)	56.6 (10.5)	662.2 (9.6)	411.0 (18.0)	232.8 (52.8)	518.0 (42.8)

TABLE III: Code coverage under the fixed testing budget. Values are reported as mean (standard deviation), with standard deviations shown in green and the best mean in each row shown in bold.

Application	Line Coverage (%)					Branch Coverage (%)				
	WebExplor	QExplore	WebQT	WebRLED	WEBCOVER	WebExplor	QExplore	WebQT	WebRLED	WEBCOVER
Agilefant	33.00 (16.45)	6.00 (0.00)	30.80 (3.42)	30.20 (5.31)	32.40 (3.85)	19.40 (11.78)	1.00 (0.00)	19.80 (1.79)	16.40 (4.34)	20.00 (2.12)
Gadael	45.87 (3.26)	36.42 (0.00)	45.20 (0.84)	44.31 (1.50)	46.49 (1.01)	18.60 (5.02)	5.84 (0.00)	17.47 (0.49)	16.00 (1.48)	18.53 (1.29)
PetClinic	49.37 (3.73)	53.12 (0.00)	54.62 (8.97)	71.51 (1.69)	72.02 (3.47)	12.00 (6.71)	15.00 (0.00)	36.78 (6.39)	61.79 (1.60)	63.21 (12.54)
SplittyPie	48.99 (3.33)	78.84 (0.25)	68.39 (0.23)	78.38 (0.05)	65.05 (2.93)	25.89 (4.63)	54.96 (0.38)	49.35 (0.97)	54.19 (0.19)	46.27 (2.92)

TABLE IV: Effective-step rate under the fixed testing budget. Values are reported as mean (standard deviation).

Application	WebExplor	QExplore	WebQT	WebRLED	WEBCOVER
4ga Boards	49.6 (9.7)	68.7 (0.2)	73.7 (9.1)	49.5 (36.2)	89.5 (7.5)
Agilefant	19.6 (7.4)	21.9 (3.5)	61.3 (3.1)	61.1 (5.2)	68.6 (5.3)
GitHub	49.0 (11.1)	92.0 (2.6)	73.8 (1.4)	50.8 (18.0)	82.6 (6.4)
Gadael	11.6 (1.9)	7.4 (0.7)	71.7 (3.4)	25.5 (10.9)	85.3 (6.6)
Nextcloud	53.7 (12.1)	29.3 (2.0)	73.9 (6.2)	73.8 (9.4)	94.3 (2.3)
Odoo	79.5 (15.2)	73.7 (7.5)	85.4 (1.0)	84.5 (4.8)	93.8 (5.1)
PetClinic	2.3 (0.5)	1.8 (0.0)	12.0 (2.2)	28.8 (9.9)	26.2 (3.6)
RealWorld	6.2 (0.6)	31.1 (3.8)	20.4 (1.0)	32.0 (16.0)	44.3 (1.4)
SplittyPie	9.5 (1.5)	0.5 (0.0)	46.5 (6.5)	72.7 (9.9)	60.3 (4.2)
TimeOff	5.0 (0.8)	85.9 (0.2)	36.7 (1.6)	42.2 (8.3)	45.8 (2.7)

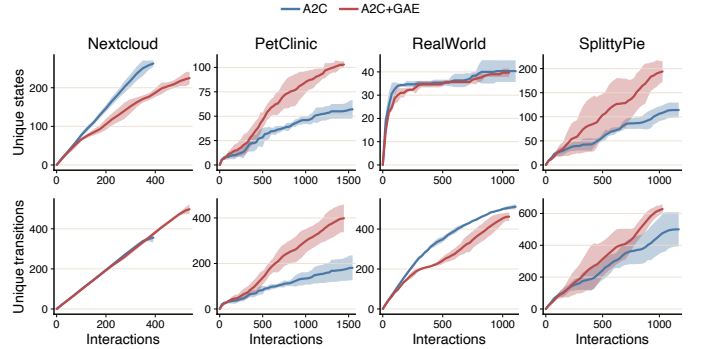


Fig. 5: Effect of GAE

B. RQ2: GAE Mechanism

This section investigates whether GAE-based multi-step advantage estimation improves the optimization stability of WEBCOVER while keeping the hard marginal coverage reward, action mask, state/action representations, network architecture, and testing budget unchanged.

The hard marginal coverage reward becomes increasingly sparse as exploration progresses, because interactions that revisit already covered states and transitions yield zero immediate gain. Thus, we define a signal zero-streak to denote a continuous sequence of zero reward agent receive in testing.

We compare two WEBCOVER variants on four web applications: the full configuration using GAE over 64-step rollouts and a one-step A2C variant using TD(0)-based advantage estimation. The two variants differ only in their advantage estimation and rollout configurations. We select Nextcloud, SplittyPie, RealWorld, and PetClinic for this ablation study. These subjects cover different application sizes and reward-sparsity patterns: Nextcloud and SplittyPie contain relatively

broad interaction spaces, whereas RealWorld and PetClinic are smaller applications in which repeated navigation and long zero-reward intervals can occur after the readily accessible coverage has been exhausted. [figure] presents the ablation results. Overall, GAE64 performs at least comparably to the one-step A2C variant across the four subjects. It achieves comparable results on Nextcloud and RealWorld, while providing clearer improvements on PetClinic and SplittyPie. It is because on PetClinic and SplittyPie, one-step A2C often falls into repeated navigation with long zero-gain intervals, where immediate TD updates provide weak credit to earlier actions. By using 64-step rollout and bootstrapping, GAE64 can propagate delayed coverage gains to preceding actions, leading to shorter zero-gain streaks and higher state/transition coverage. In contrast, on denser sites such as Nextcloud and RealWorld, one-step A2C already receives frequent useful feedback, so longer rollouts provide less benefit and may smooth out short-term signals.

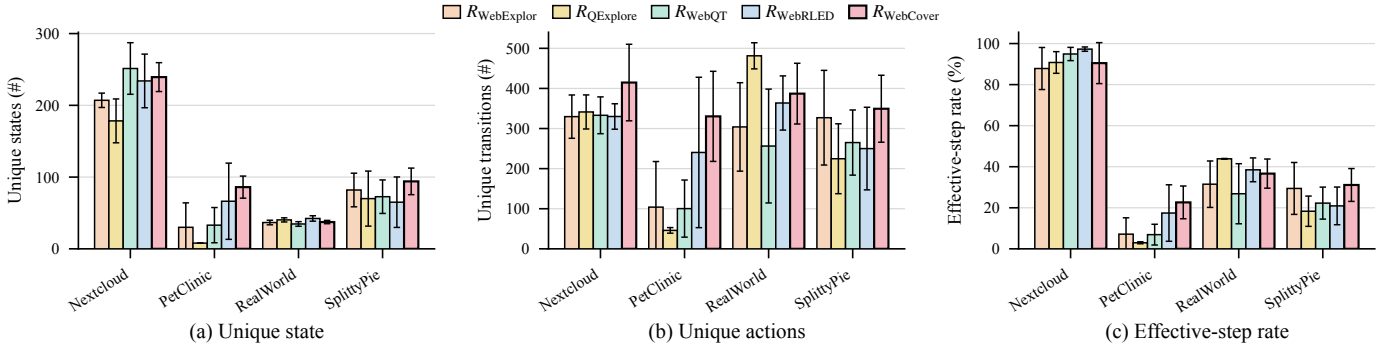


Fig. 6: Effect of Reward Design

To illustrate the behavioral difference between the two variants, we further inspect their detailed execution traces on PetClinic. Figure 5 shows the zero streak and transition diagram changed by interaction. In PetClinic, the one-step A2C variant repeatedly navigates among common pages such as /, /vets.html, /owners/find, and /oups. Between interactions 32 and 187, the illustrated one-step A2C run performs 156 consecutive interactions without discovering a new state or transition. In comparison, the GAE64 runs report a substantially shorter zero-reward-streak statistic of 68.0, together with higher state and transition coverage. Therefore, the GAE mechanism is particularly effective in sparse-reward settings and improves the robustness.

Answer to RQ2: GAE helps WEBCOVER handle sparse rewards under the hard marginal coverage objective and improves its robustness across diverse web applications.

C. RQ3: Reward Design

To isolate the effect of reward design, Figure 6 compares five reward designs under the same state/action abstraction and A2C-based optimization backbone. The compared rewards correspond to representative reward styles used in prior web-testing systems: count-decay curiosity ($R_{WebExplor}$), manually weighted testing proxies (R_{WebQT} and R_{WebLED}), an execution-count-based QExplore-style reward ($R_{QExplore}$), and our hard marginal coverage reward ($R_{Submodular}$). We report unique states, unique transitions, total interactions, and redundant-interaction rate.

Overall, the submodular marginal reward achieves the strongest transition coverage on all four subjects shown in the figure. This trend is especially clear on PetClinic and RealWorld, where the submodular reward discovers substantially more unique transitions than the count-decay and proxy rewards. The result suggests that directly optimizing marginal coverage better aligns the training signal with the testing objective: once a state-transition has already been covered, repeating it no longer provides reward, so the agent is pushed toward interactions that can expand the covered set.

The redundant-interaction results provide a complementary view. On Nextcloud, all reward designs exhibit relatively low redundancy, suggesting that the application exposes many

actions with immediate coverage potential. In contrast, trace inspection on PetClinic and SplittyPie reveals small navigation regions and recurrent form workflows, in which rewards that continue to assign positive value to frequently repeated behavior may reinforce local loops. The submodular reward reduces this effect by assigning zero marginal gain to already covered transitions. Although redundancy is not eliminated entirely because some repeated actions are necessary bridges to later coverage, the marginal reward makes such repetition useful only when it leads to downstream coverage gains.

We further examine the PetClinic traces to illustrate the failure mode of the QExplore-style reward. Across the three PetClinic QExplore-style runs, the agent covers only 8 states in every run and 39–53 unique transitions, while executing more than 1,500 interactions. The corresponding redundant-interaction rate ranges from 96.6% to 97.5%. The executed transitions are also highly concentrated: in two runs, the two most frequent transition triples account for more than 80% of all interactions. For example, one run repeatedly executes the two transitions between states S_2 and S_4 670 and 578 times, respectively. This behavior indicates that the QExplore-style count signal can become concentrated on a small set of valid navigation actions on PetClinic. By comparison, the submodular runs cover 73–103 states and 262–460 unique transitions, and their top repeated transitions occupy a much smaller fraction of the budget. This case study is consistent with the aggregate results: the QExplore-style count signal can become concentrated on a small set of valid actions, whereas the hard marginal reward suppresses immediate rewards for already covered local transitions and preserves learning pressure for interactions that expand downstream coverage.

Answer to RQ3: WEBCOVER with its marginal-gain reward show effectiveness and generalizability among all baselines both at the system level and at reward level.

VIII. THREATS TO VALIDITY

State and action abstraction. WEBCOVER’s objective is defined over abstract web states and action signatures. Over-merging can hide useful behavior, while over-splitting can inflate coverage. We mitigate this by using one canonicaliza-

tion pipeline for posthoc coverage metrics, but the abstraction remains an implementation choice.

Benchmark representativeness. The evaluated applications may not cover all architectures, UI frameworks, authentication flows, and data dependencies found in modern web applications. We use both benchmark and larger real-world applications to reduce this threat.

Randomness and repeated runs. RL-based testing is inherently stochastic due to action sampling and nondeterministic application behavior. We therefore repeat each experiment with multiple runs and report aggregate results.

Baseline fairness. Baselines differ in state abstraction, action extraction, and reward design. Whenever possible, we run all methods under the same time or step budget and recompute coverage metrics using the same posthoc canonicalizer. We distinguish official tool runs from local style-reimplementations.

IX. RELATED WORK

A. Web Test Generation and Model-Based Exploration

Model-based web testing is a major paradigm for AWTG, which build an abstract navigation model to describe the behaviors of a web application, and then derive test cases from the model [18], [19], [20], [21], [22], [23]. For example, Crawljax [18] addresses the limitations of URL-based crawling in Ajax applications by executing DOM events in a browser and incrementally inferring a state-flow graph from observed UI changes. Invariant-based testing then uses explored states and transitions to check application-level properties [19]. Later techniques improve model construction and state abstraction: FEEDEX [20] uses feedback such as structural diversity and coverage information to guide exploration, while FRAGGEN [24] represents pages as fragments to avoid overly coarse whole-page abstractions. Overall, model-based techniques rely on constructed models to generate tests, so their effectiveness is limited by model quality and completeness. In contrast, WEBCOVER is model-free: it learns directly from browser interactions and uses marginal coverage gain to guide action selection within a fixed testing budget.

B. Reinforcement Learning for Web and GUI Testing

Reinforcement learning has been used in GUI and web interaction tasks. In Android testing, QBE [25] applies Q-learning to GUI exploration, curiosity-driven testing uses state similarity and novelty rewards [8], and ARES studies deep RL for black-box Android testing [26]. Other GUI testing systems use goal-driven exploration, stochastic model-based exploration, or multi-objective search [27], [28], [29]. RL has also been used to operate web interfaces, such as workflow-guided exploration and DOM-Q-NET [30], [31]. However, these systems mainly target Android GUI exploration or web task completion, rather than fixed-budget web testing coverage.

For web testing, WebExplor [1] uses tabular Q-learning with a transition-frequency curiosity reward and constructs an automaton to guide exploration when the agent stalls.

QExplore [4] also builds on Q-learning and combines guided search with input-generation logic. More recent systems use richer state/action representations and reward designs: WebQT [2] combines locality, changed elements, transition frequency, and unexplored-action information in a manually designed reward model, while WebRLED [3] learns action values from neural page/action representations with episodic and global reward signals. These systems demonstrate the benefit of adaptive exploration, but their rewards remain novelty proxies, count-decay signals, or manually weighted local components. In contrast, WEBCOVER uses the strict marginal gain of a non-additive state-transition coverage objective as the learning reward.

C. Curiosity, Novelty, and Submodular Reward Design

Curiosity and intrinsic motivation are widely used to encourage exploration in sparse-reward environments. Count-based methods reward rare states or state-action pairs [32]; prediction-error methods reward states that are hard to predict [7]; episodic curiosity uses memory or reachability to promote within-episode novelty [33]; and large-scale agents such as NGU and Agent57 combine episodic and lifelong novelty signals [34], [35]. RL-based testing rewards often follow this intuition through transition-frequency decay, state-memory novelty, or action-diversity terms. However, such rewards may still assign positive curiosity reward to repeated transitions even when they no longer increase testing coverage. Submodular objectives formalize the diminishing-return structure of coverage and diversity [36]. Adaptive submodularity extends this idea to sequential decision-making under partial observations [37], while submodular RL studies non-additive, history-dependent trajectory rewards [12]. Standard RL and actor-critic methods provide the optimization machinery for learning from sampled trajectories [11], [38], [39], [14]. WEBCOVER brings this perspective to web testing by defining reward as the strict marginal gain of covered canonical states and resolved transitions, thereby aligning the per-step RL signal with the fixed-budget coverage metric used for evaluation.

X. CONCLUSION

This paper presents WEBCOVER, an RL-based web testing framework that treats fixed-budget web exploration as submodular state/action coverage. Instead of relying on local novelty proxies or manually weighted reward indicators, WEBCOVER defines the testing reward as the strict marginal gain of covered states and executed transitions. A masked online actor-critic policy optimizes this hard coverage objective over variable web action sets, while a coverage signal improves learning. Our evaluation will show whether this objective-level formulation improves exploration efficiency and downstream testing outcomes across web applications.

XI. DATA AVAILABILITY

All experimental data are available at <https://doi.org/10.5281/zenodo.21104041>.

REFERENCES

- [1] Y. Zheng, Y. Liu, X. Xie, Y. Liu, L. Ma, J. Hao, and Y. Liu, "Automatic web testing using curiosity-driven reinforcement learning," 2021. [Online]. Available: <https://arxiv.org/abs/2103.06018>
- [2] X. Chang, Z. Liang, Y. Zhang, L. Cui, Z. Long, G. Wu, Y. Gao, W. Chen, J. Wei, and T. Huang, "A reinforcement learning approach to generating test cases for web applications," in *2023 IEEE/ACM International Conference on Automation of Software Test (AST)*, 2023, pp. 13–23.
- [3] Z. Gu, C. Liu, G. Wu, Y. Zhang, C. Yang, Z. Liang, W. Chen, and J. Wei, "Deep reinforcement learning for automated web gui testing," 2025. [Online]. Available: <https://arxiv.org/abs/2504.19237>
- [4] S. Sherin, A. Muqet, M. U. Khan, and M. Z. Iqbal, "Qexplore: An exploration strategy for dynamic web applications using guided search," *Journal of Systems and Software*, vol. 195, p. 111512, 2023.
- [5] Y. Fan, S. Wang, Z. Fei, Y. Qin, H. Li, and Y. Liu, "Can cooperative multi-agent reinforcement learning boost automatic web testing? an exploratory study," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 14–26.
- [6] S. Wang, S. Wang, Y. Fan, X. Li, and Y. Liu, "Leveraging large vision-language model for better automatic web gui testing," in *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2024, pp. 125–137.
- [7] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2017, pp. 16–17.
- [8] M. Pan, A. Huang, G. Wang, T. Zhang, and X. Li, "Reinforcement learning based curiosity-driven testing of android applications," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSA 2020, 2020, pp. 153–164.
- [9] L. Mariani, M. Pezzè, O. Riganelli, and M. Santoro, "Automatic testing of gui-based applications," *Software Testing, Verification and Reliability*, vol. 24, no. 5, pp. 341–366, 2014.
- [10] Y. Fan, S. Wang, S. Wang, Y. Liu, G. Wen, and Q. Rong, "A comprehensive evaluation of q-learning based automatic web gui testing," in *2023 10th International Conference on Dependable Systems and Their Applications (DSA)*. IEEE, 2023, pp. 12–23.
- [11] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [12] M. Prajapat, M. Mutny, M. N. Zeilinger, and A. Krause, "Submodular reinforcement learning," 2024. [Online]. Available: <https://arxiv.org/abs/2307.13372>
- [13] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," *Advances in neural information processing systems*, vol. 12, 1999.
- [14] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proceedings of the 33rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 48, 2016, pp. 1928–1937.
- [15] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.
- [16] R. S. Sutton, "Learning to predict by the methods of temporal differences," *Machine learning*, vol. 3, no. 1, pp. 9–44, 1988.
- [17] N. Metropolis and S. Ulam, "The monte carlo method," *Journal of the American statistical association*, vol. 44, no. 247, pp. 335–341, 1949.
- [18] A. Mesbah, A. van Deursen, and S. Lenselink, "Crawling ajax-based web applications through dynamic analysis of user interface state changes," *ACM Transactions on the Web*, vol. 6, no. 1, 2012.
- [19] A. Mesbah, A. van Deursen, and D. Roest, "Invariant-based automatic testing of modern web applications," *IEEE Transactions on Software Engineering*, vol. 38, no. 1, pp. 35–53, 2012.
- [20] A. M. Fard and A. Mesbah, "Feedback-directed exploration of web applications to derive test models," in *2013 IEEE 24th International Symposium on Software Reliability Engineering (ISSRE)*, 2013, pp. 278–287.
- [21] A. Marchetto, P. Tonella, and F. Ricca, "State-based testing of ajax web applications," in *2008 1st International Conference on Software Testing, Verification, and Validation*, 2008, pp. 121–130.
- [22] S. Artzi, A. Kiezun, J. Dolby, F. Tip, D. Dig, A. Paradkar, and M. D. Ernst, "Finding bugs in dynamic web applications," in *Proceedings of the 2008 International Symposium on Software Testing and Analysis*, ser. ISSTA '08, 2008, pp. 261–272.
- [23] S. Artzi, J. Dolby, S. H. Jensen, A. Møller, and F. Tip, "A framework for automated testing of javascript web applications," in *2011 33rd International Conference on Software Engineering (ICSE)*, 2011, pp. 571–580.
- [24] R. K. Yandrapally and A. Mesbah, "Fragment-based test generation for web apps," *IEEE Transactions on Software Engineering*, vol. 49, no. 3, pp. 1086–1101, 2023.
- [25] Y. Koroglu, A. Sen, O. Muslu, Y. Mete, C. Ulker, T. Tanriverdi, and Y. Donmez, "Qbc: Qlearning-based exploration of android applications," in *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*, 2018, pp. 105–115.
- [26] A. Romdhana, A. Merlo, M. Ceccato, and P. Tonella, "Deep reinforcement learning for black-box testing of android apps," *ACM Transactions on Software Engineering and Methodology*, vol. 31, no. 4, 2022.
- [27] D. Lai and J. Rubin, "Goal-driven exploration for android applications," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019, pp. 115–127.
- [28] K. Mao, M. Harman, and Y. Jia, "Sapienz: Multi-objective automated testing for android applications," in *Proceedings of the 25th International Symposium on Software Testing and Analysis*, ser. ISSTA 2016, 2016, pp. 94–105.
- [29] T. Su, G. Meng, Y. Chen, K. Wu, W. Yang, Y. Yao, G. Pu, Y. Liu, and Z. Su, "Guided, stochastic model-based gui testing of android apps," in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, ser. ESEC/FSE 2017, 2017, pp. 245–256.
- [30] E. Z. Liu, K. Guu, P. Pasupat, T. Shi, and P. Liang, "Reinforcement learning on web interfaces using workflow-guided exploration," 2018. [Online]. Available: <https://arxiv.org/abs/1802.08802>
- [31] S. Jia, J. Kiros, and J. Ba, "Dom-q-net: Grounded rl on structured language," 2019. [Online]. Available: <https://arxiv.org/abs/1902.07257>
- [32] M. G. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, and R. Munos, "Unifying count-based exploration and intrinsic motivation," in *Advances in Neural Information Processing Systems*, 2016, pp. 1471–1479.
- [33] N. Savinov, A. Raichuk, R. Marinier, D. Vincent, M. Pollefeys, T. P. Lillicrap, and S. Gelly, "Episodic curiosity through reachability," in *International Conference on Learning Representations*, 2019.
- [34] A. P. Badia, P. Sprechmann, A. Vitvitskiy, D. Guo, B. Piot, S. Kapturowski, O. Tieleman, M. Arjovsky, A. Pritzel, A. Bolt, and C. Blundell, "Never give up: Learning directed exploration strategies," 2020. [Online]. Available: <https://arxiv.org/abs/2002.06038>
- [35] A. P. Badia, B. Piot, S. Kapturowski, P. Sprechmann, A. Vitvitskiy, Z. D. Guo, and C. Blundell, "Agent57: Outperforming the atari human benchmark," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 507–517.
- [36] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions-i," *Mathematical Programming*, vol. 14, no. 1, pp. 265–294, 1978.
- [37] D. Golovin and A. Krause, "Adaptive submodularity: Theory and applications in active learning and stochastic optimization," *Journal of Artificial Intelligence Research*, vol. 42, pp. 427–486, 2011.
- [38] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3, pp. 279–292, 1992.
- [39] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.